

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра экономической математики, информатики и статистики (ЭМИС)

НАСЛЕДОВАНИЕ

Отчёт по лабораторной работе по дисциплине «Объектно-ориентированное
программирование»

Студент гр.594-2

 А. А. Танцеров

“ 19 ” Октября 2025 г.

Руководитель канд. техн. наук, доцент
каф. ЭМИС ТУСУР

_____ Я. В. Костелей

“ _____ ” _____ 2025 г.

Томск 2025

Лабораторная работа №4

«Наследование»

Цель работы: изучение принципов объектно-ориентированного программирования в C++, включая создание иерархии классов, перегрузку операторов и методов, а также работу с динамической памятью.

Наследование – это метод определения класса, при котором, кроме объявляемых методов и полей, происходит копирование методов и полей из другого класса. Класс, у которого «наследуют» методы и поля, называется родителем или базовым классом, а класс реципиент – потомком или ребенком, или производным классом. Именно, здесь и проявляются функциональные возможности применения модификатора `protected`. Если поля или методы базового класса помечены модификатором `protected`, то данные поля не будут «видимы» для внешнего кода, но класс, который наследуется от базового класса, будет иметь к ним доступ внутри себя (внутри класса). Этот механизм также инкапсулирует логику внутри класса и его потомков при наследовании. Но стоит отметить, что члены базового класса с модификатором доступа `private` будут добавляться в производный класс, но доступ к ним внутри производного класса не будет предоставлен. Как было сказано ранее, можем при определении класса успешно наследовать поля и методы базового класса. Все бы было просто, если бы не модификаторы наследования: класс потомок может наследоваться от базового класса по разным правилам, которые определяют то, с каким модификатором члены базового класса войдут в класс потока. То есть, кроме того, что до этого нужно было следить за тем, как будут видны поля и методы класса во внешнем коде, то теперь необходимо продумывать, как в классе-потомке будут видны поля базового класса.

То есть при всех типах наследования `private`-члены базового класса недоступны, при `public`-наследовании остальные члены сохраняют свой модификатор, при `private`- и `protected`-наследовании остальные члены класса приобретают соответствующий типу наследования модификатор.

Задание. Создайте производный класс, который бы позволил прикреплять дополнительное пояснение к приему (строка – `char*`, например, рекомендации по тому, что нужно принимать до еды, или нужно разжевывать или глотать капсулу). Создать метод для установки этого значения. Учесть вывод этого поля на консоль. При необходимости измените модификаторы доступа у базового класса.

На первом этапе был расширен класс `Medicine` путём перегрузки ключевых операторов и методов. Реализован дополнительный конструктор копирования с флагом копирования плана приёма, позволяющий создавать объекты для следующего дня на основе данных предыдущего. Перегружен оператор `<<` для удобного вывода информации в консоль, оператор `[]` для установки факта приёма через индексацию, а также метод `needToTake()` с параметром по умолчанию для проверки как конкретного времени, так и общего соответствия плану.

На втором этапе создан производный класс `Medicines` (правильнее было бы назвать `MedicineWithNote` для семантической ясности), унаследованный от `Medicine`. Класс дополнен динамическим полем `char* note` для хранения дополнительных пояснений к приёму лекарства. Реализованы: конструктор с параметрами (название, день, примечание с значением по умолчанию `nullptr`), деструктор для освобождения памяти примечания, метод `setNote()` для установки/изменения пояснений, геттер `getNote()` и перегруженный метод `print()`, который выводит полную информацию включая примечание (при его наличии). Для корректной работы производного класса поля базового класса были переведены в режим `protected` доступа. Протестирована работа всех методов с учётом системы исключений, включая обработку пустых строк в примечаниях и корректное копирование объектов.

Заголовочный файл производного класса представлен на листинге 1. Файл `cpp` представлен на листинге 2.

```

#pragma once
#include "medicine.h"

class Medicines : public Medicine {
private:
    char* note;
public:
    Medicines(const char* name, int day, const char* note = nullptr);
    ~Medicines();
    void setNote(const char* info);
    void print() const;
    const char* const getNote() const;
};

```

Листинг 1 – Код заголовочного файла

```

#include "Medicines.h"
#include <iostream>
using namespace std;

Medicines::Medicines(const char* name, int day, const char* note ) :
Medicine(name, day) {
    if (note && strlen(note) > 0) {
        this->note = new char[strlen(note) + 1]; // TODO ñíéòèñü ià ääéíî
        strcpy_s(this->note, strlen(note) + 1, note);
    }
    else {
        const char* text = "Äî ääû, äëîäàöü.";
        this->note = new char[strlen(text) + 1];
        strcpy_s(this->note, strlen(text) + 1, note);
    }
}

Medicines::~~Medicines(){
    delete[] note;
}

const char* const Medicines::getNote() const {
    return note;
}

void Medicines::setNote(const char* info) {
    delete[] note;
    note = nullptr;
    if (info) {
        note = new char[strlen(info) + 1];
        strcpy_s(note, strlen(info) + 1, info);
    }
    note = nullptr;
}

void Medicines::print() const {
    Medicine::print();
    if (note) {
        cout << note << endl;
    }
}

```

Листинг 2 – Код файла с реализацией классов

Вывод: изучены механизмы объектно-ориентированного программирования в C++, включая перегрузку операторов и методов.